

```
import React from 'react';

export const generateStaticParams = async () => {
  const res = await fetch("http://localhost:5000/books");
  const books = await res.json();

  return books.slice(0,3).map(book => ({bookId: book.id}));
}

const BookDetailsPage = async ({params}) => {
  const {bookId} = await params;

  const res = await fetch(`http://localhost:5000/books/${bookId}`);
  const book = await res.json();
  return (
    <div>
      <h2>Book Details</h2>
      <p>Book Name: {book.title}</p>
      <p>Author: {book.author}</p>
      <p>Price: {book.price}</p>
    </div>
  );
};

export default BookDetailsPage;
```

generateStaticParams() দিয়ে যদি **/books/1** আগে থেকেই **generate** করা হয়, তাহলে **user** যখন **/books/1** **visit** করবে তখন **page()** আবার চালিয়ে ওই **book**-এর জন্য **fetch** করার দরকার নেই।

তাহলে তোমার **code**-এ দুইবার **fetch** কেন দেখা যাচ্ছে?

তোমার এই **code**:

```
generateStaticParams = async () => {  
  const res = await fetch("http://localhost:5000/books");  
  const books = await res.json();  
  
  return books.slice(0,3).map(book => ({  
    bookId: book.id  
  }));  
}
```

এখানে প্রথম **fetch**:

```
fetch("http://localhost:5000/books")
```

এর উদ্দেশ্য **book**-এর **details** নেওয়া না।

এটা শুধু বের করছে:

```
1  
2  
3
```

অর্থাৎ:

```
[  
  { bookId: 1 },  
  { bookId: 2 },  
  { bookId: 3 }  
]
```

যাতে **Next.js** জানে:

এই ৩টা **page** generate করতে হবে:

```
/books/1  
/books/2  
/books/3
```

তারপর static generation-এর সময় `page()`-ও execute হবে।

```
const res = await fetch(
  `http://localhost:5000/books/${bookId}`
);
```

`bookId = 1` হলে:

`/books/1`

এর **actual data** আনবে:

```
{
  "id": 1,
  "title": "Book A",
  "author": "XYZ",
  "price": 500
}
```

তারপর সেই data দিয়ে HTML/page তৈরি হবে।

তাই **actual flow**:

BUILD TIME

`generateStaticParams()`

↓

GET /books

↓

[1, 2, 3]

↓

Next.js বলে:

এই ৩টা route generate করব

↓

`page({bookId: 1})`

↓

GET /books/1

↓

Book 1 data

↓

/books/1 page তৈরি

একইভাবে 2 এবং 3।

এরপর **USER /books/1** এ গেলে

আবার **/books/1** fetch করার কথা নয় যদি সেটা successfully static-generated page হয়।

সে আগে থেকে তৈরি করা page-টাই পাবে।

USER

↓

/books/1

↓

already generated static page

↓

SHOW

শুধু একটা জিনিস মনে রাখো:

generateStaticParams() নিজে page-এর HTML/data generate করে না।
এটা কোন কোন **dynamic route generate** করতে হবে তার **list** দেয়।

তারপর **page()** **build time**-এ সেই route-এর data fetch করে page তৈরি করে।

তাই তোমার code-এ **/books/1**-এর জন্য fetch দুইবার নয়:

- **GET /books** → কোন IDs generate করব সেটা জানার জন্য
- **GET /books/1** → build time-এ **/books/1** page-এর actual data নেওয়ার জন্য
- User **/books/1** visit করলে → **static page serve** হবে, আবার fetch করার প্রয়োজন নেই।

generateStaticParams()

return করে:

```
[  
  { bookId: 1 },  
  { bookId: 2 },  
  { bookId: 3 }  
]
```

তাহলে build time-এ:

```
/books/1 → data fetch → page generate  
/books/2 → data fetch → page generate  
/books/3 → data fetch → page generate
```

তাই user:

```
/books/1  
/books/2  
/books/3
```

এ গেলে আগে থেকে **generate** করা **static page** পাবে।

কিন্তু user:

```
/books/4
```

এ গেলে কী হবে সেটা নির্ভর করে তোমার dynamic route configuration-এর ওপর (**dynamicParams** ইত্যাদি)।

Default behavior-এ সাধারণত Next.js নতুন **param**-এর জন্য **page generate** করতে পারে:

```
/books/4  
↓  
page() চালাবে  
↓  
fetch("/books/4")  
↓  
page তৈরি
```

তাই তোমার mental model মোটামুটি:

generateStaticParams
↓
1, 2, 3 আগে থেকেই generate
↓
User → 1,2,3
↓
আগের তৈরি page

User → 4
↓
আগে generate করা ছিল না
↓
নতুন করে generate/fetch হতে পারে

generateStaticParams() বলে:

"Dynamic route-এর কোন কোন parameter-এর জন্য আগে থেকেই page তৈরি করব?"

generateStaticParams() থেকে { bookId: ... } return করার লাভ হলো Next.js জানতে পারে [bookId] route-এর কোন কোন values-এর জন্য static page generate করতে হবে।